Parallel Computing Project Report

Lukas Strobel
Wiktor Andrzejewski
p220260@campus.fct.unl.pt
p220140@campus.fct.unl.pt

ABSTRACT

We study the performance of a PySpark pipeline on the NYC Yellow Taxi Trip Records: three mandatory tasks (cleaning, temporal demand, zone statistics) and three optional ones (fare anomalies, origin–destination flows, demand features). Following a profile–hypothesise–optimise–measure loop, we establish a baseline with adaptive execution and automatic broadcast joins disabled, and use Spark UI metrics and physical plans to locate bottlenecks. Replacing sort-merge joins with broadcast joins reduces shuffle write by 99.7 % and wall time by 59 % in the fare anomaly task, and shuffle write by 20 % and wall time by 14 % in the demand features task. Repeated scans and many tiny shuffle tasks remain as open bottlenecks.

KEYWORDS

Apache Spark, performance engineering, shuffle, broadcast join, NYC taxi data

1 INTRODUCTION

This report studies the performance of a PySpark analytics pipeline on the NYC Taxi & Limousine Commission (TLC) Yellow Taxi Trip Records [1]. We use the monthly Parquet files for 2025, together with the `taxi_zone_lookup.csv`, which maps each `LocationID` to a borough, zone, and service zone. The pipeline consists of three mandatory tasks: **M1** cleans the data and reports per-year removal counts per validity criterion, **M2** computes demand profiles per borough, weekday, and hour, and **M3** computes monthly zone statistics with window functions (volume rank, rolling 3-month share, and deviation flags). In addition, we implement three optional tasks: **O1** (fare anomaly detection per zone and hour), **O2** (origin–destination flow analysis), and **O3** (demand forecasting features with a self-join lag). We selected them because they stress different parts of Spark's execution: joining per-bucket statistics back to the full trip table, high-cardinality (PU, DO, month) keys, two lookup joins and windows, and a self-join that adds extra shuffles.

This workload is interesting from a performance standpoint because it combines several known Spark cost drivers. All tasks start from the same cleaned trip table, so this intermediate result may be recomputed many times. The aggregations use fine-grained keys, which can make shuffles large relative to the input. Group and window keys are few and unevenly distributed, which can cause skew. Finally, the joins with a tiny lookup table are candidates for broadcasting.

Following the profile → hypothesis → optimize → measure loop, we first establish a baseline with adaptive query execution and automatic broadcast joins disabled (Section 3). We then apply targeted optimizations (Section 4) and evaluate scaling with data size and cluster size (Section 5).

2 PIPELINE

All tasks are implemented with the PySpark DataFrame API [3] rather than Spark SQL. The pipeline is a chain of column expressions, which is easier to compose and test in this form, and both APIs compile to the same Catalyst plan. The pipeline reads the Parquet files given by the input path, and the dataset scale is selected by parameter.

M1 (ingestion and cleaning). The Parquet files are read with an explicit target schema[2], and columns are cast to these types afterwards. Each validity rule becomes a Boolean flag column: `fare_amount` ≤ 0, `trip_distance` ≤ 0,, `passenger_count` $= 0$ or > 6, dropoff $\leq$ pickup, and zone IDs outside 1–263. Nulls in the rule columns are counted as a separate criterion. A single `groupBy(year)` pass sums all flags to give the per-year removal counts. Records with any flag set are removed, which yields `clean_trips`, the common input of all other tasks.

M2 (temporal demand). `clean_trips` is joined with the zone lookup on `PULocationID` and grouped by (borough, day of week, hour). The aggregates are trip count and mean fare, duration (from the timestamps), and distance. The result is written as Parquet partitioned by borough.

M3 (zone statistics). Trips are first aggregated per (zone, month) to obtain the mean and standard deviation of fare and distance. Window functions on this small table then compute (i) the volume rank within the borough and month, (ii) the rolling 3-month share within the borough, using a range frame over a month index, and (iii) a flag for months whose volume deviates from the rolling mean of the three preceding months by more than 1.5 standard deviations.

O1 (fare anomalies). The fare per mile is computed per trip, and mean and σ are computed per (`PULocationID`, hour). These statistics are joined back to the trips, and the borough is attached through a second join. Trips above mean + 3σ are flagged, and anomaly rates are aggregated by borough, payment type, and weekday/weekend.

O2 (origin–destination flows). The pipeline counts trips per (PU, DO, month) and joins the zone lookup twice (pickup and dropoff). A window over (PU, month) gives each route's share of the outbound trips. A window over (borough, month) ranks the routes to select the top 20. The Q1 and Q3 shares are then compared through a self-join on the route.

O3 (demand features). Trips are counted per (zone, date, hour). The historical mean and variance are computed per (zone, weekday, hour). The 4-week lag is modeled as a self-join on a copy of the table with the date shifted by 28 days. The demand index (count ÷ mean) is flagged above 2.0, and the result is annotated with `service_zone`.

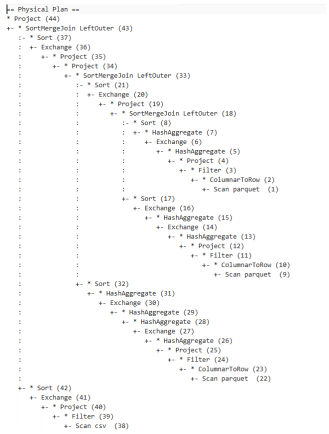


Figure 1: Formatted Physical Plan for Task O3

Figure 1 shows the formatted physical plan of O3 in the baseline, which is also the starting point of the bottleneck analysis in Section 3.

3 PROFILING AND BASELINE

To establish a baseline, all workloads first ran with plain PySpark APIs, with Adaptive Query Execution (AQE) and automatic broadcast joins disabled (`spark.sql.adaptive.enabled = false`, `spark.sql.autoBroadcastJoinThreshold = -1`) so that the underlying bottlenecks are visible.

3.1 Experimental Setup

Experiments ran on Apache Spark 4.2 in a standalone Docker setup with one master and one worker (2 cores, 2 GB memory). The dataset comprised the 12 monthly Parquet files of 2025 and the zone lookup table; the Spark UI reports 10.6 MiB of input per scan of the trip table. We used the Spark UI, `explain`, and a profiler based on the UI REST API that records wall time, jobs, stages, tasks, shuffle bytes, and spill for each profiled block.

3.2 Baseline Execution Analysis and Identified Bottlenecks

Execution waves and parallelism. The Event Timeline (Figure 2) shows a block-like step pattern. With few executor cores, Spark processes tasks in sequential waves; in stages with the default 200 shuffle partitions, many waves are needed. This amplifies any per-task latency or scheduling delay.



Figure 2: Task event timeline showing execution waves.

Stage outliers and disk spilling. In O2 (origin–destination flows), most stages completed within 2 to 6 seconds, but two stages took **34 seconds** and **57 seconds** (Figure 3). The task summary metrics of these stages show memory pressure: some stages recorded **96.0 MiB of memory spill** and **31.8 MiB of disk spill**, which adds disk I/O and CPU stall time. The high-cardinality grouping over (*PULocationID*, *DOLocationID*) pairs and the window rank computations are the likely cause.

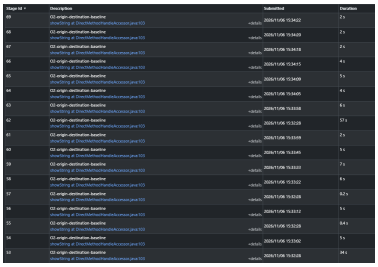


Figure 3: Stage durations of the O2 baseline with outliers of 34 s and 57 s (Stages 53 and 62).

Shuffle volume from forced SortMergeJoins. Without automatic broadcast, joins between the trip data and the 12.0 KiB lookup table run as `SortMergeJoin`. Each requires a full `Exchange` (hash partitioning into 200 partitions) and a sort on both inputs. Stages 47 and 49 of O1 (Figure 4) shuffle the trip table for a tiny static table, which generated over 600 MiB of shuffle read/write per job.

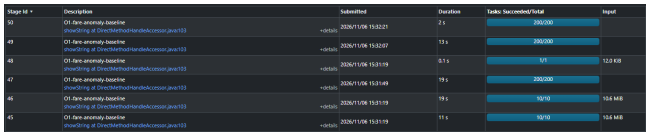


Figure 4: Stage metrics of the O1 baseline: Stages 46 and 47 take 19 s each, driven by wide shuffles.

Redundant Parquet scans. The physical plan of O3 contains three separate `FileScan parquet` nodes (`*(1)`, `*(3)`, `*(8)`) over the same files (Figure 5): one for the daily aggregation, one for the 4-week lag, and one for the historical statistics. Each re-applies the same filters, so the input is read three times (31.9 MiB instead of 10.6 MiB), and three stages (113, 115, 116) each write 13.2 MiB of shuffle data.

3.3 Summary of Baseline Performance Metrics

Table 1 lists the disk spill measured in the baseline runs.

Table 1: Disk spill in the baseline runs.

Workload run	Disk spill
`task-2_temporal_demand`	34.9 MiB
`task-3_zone_stats`	31.8 MiB
`O1-fare-anomaly-baseline`	19.2 MiB

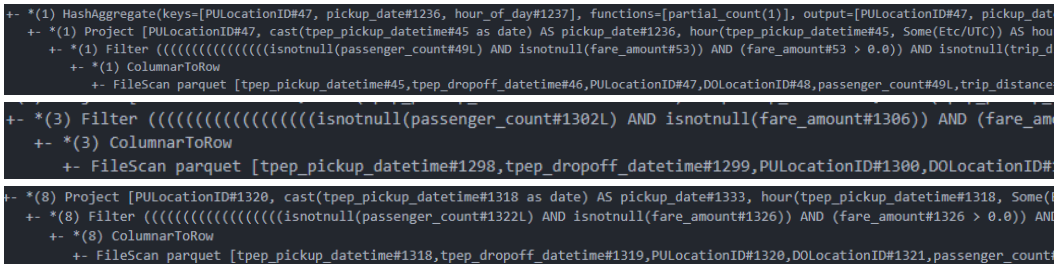


Figure 5: Three scans of the same Parquet files in the O3 baseline plan (nodes `*(1)`, `*(3)`, `*(8)`).

4 OPTIMISATION STUDY

Each optimisation follows the loop of Section 3: bottleneck from the baseline profile, hypothesis, one change, and a re-measurement on the same data (12 files of 2025), cluster, and configuration. All three optimisations replace shuffle-based joins by broadcast hash joins, the most expensive pattern the baseline exposes in O1, O2, and O3.

4.1 O1: broadcasting the join inputs

(a) Bottleneck. The statistics per (zone, hour) and the zone lookup are joined back to every trip with `SortMergeJoin`, which shuffles the full trip table (Stages 46/47: 19 s each, 621 tasks, 19.2 MiB disk spill). **(b) Hypothesis.** Both join inputs are tiny compared to the trip table. Broadcasting them removes the shuffle and the sort of the large side, and with it the spill. **(c) Change.** Both joins use `F.broadcast(...)` on the small side. **(d) Result** (Figure 6, left). Wall time falls from 65.6 s to 27.0 s (2.4× faster), tasks from 621 to 44, executor run time from 119.5 s to 50.2 s, shuffle read/write by 99.7 %, and the spill disappears. The number of jobs rises from 1 to 5 and stages from 6 to 7, because Spark materialises the broadcast tables in extra jobs.

4.2 O2: broadcasting the zone lookups

(a) Bottleneck. The two lookup joins in O2 run as sort-merge joins, and Stages 53 and 62 take 34 s and 57 s with 96.0 MiB of memory spill and 31.8 MiB of disk spill (Figure 3). **(b) Hypothesis.** Joining the 265-row lookup without a shuffle and sort removes the memory pressure that causes the spill and reduces the shuffle volume. **(c) Change.** The lookup joins use `F.broadcast(...)`. **(d) Result** (Figure 6, right). Shuffle read/write drops by 83.9 % to 4.6 MB, and memory and disk spill fall to 0 B. The optimised run takes 120.8 s over 19 stages and 2624 tasks. The window ranking and the Q1/Q3 comparison still shuffle, which explains why O2 stays the slowest task.

4.3 O3: broadcasting the lag, statistics, and lookup tables

(a) Bottleneck. The three joins of O3 are sort-merge joins with eight `Exchange` nodes and nine stages. The final join shuffles the whole result by `PULocationID` (Exchange `plan_id=3133`) only to attach a 265-row table (Exchange 3142). The baseline shuffles 126.7 MB. **(b) Hypothesis.** The statistics table has at most 263×7×24 rows and the lookup 265, so both fit easily in memory. Broadcasting them removes the Exchange on the large side. The lag table is as large as the daily table and benefits less. **(c) Change.** The lag, statistics, and lookup tables are wrapped in `F.broadcast(...)`. **(d) Result** (Figure 7). Wall time falls from 112.1 s to 96.8 s (−13.6 %), executor CPU time by 27.4 %, and shuffle read/write by 19.7 % (126.7 to 101.7 MB). Stages (9) and tasks (1031) are unchanged, and the jobs rise from 1 to 3. The output size grows from 7.4 to 10.0 MB (+35.2 %); we did not investigate this.

4.4 Summary

Table 2 summarises the results. The three scans, the 200 tiny shuffle partitions, and the self-join of the lag are still present and are discussed in Section 6.

Table 2: Impact of the broadcast joins relative to the baseline (same data, same cluster). The baseline wall time of O2 is not available.

Task	Wall time (s)	Tasks	Shuffle wr.	Spill
O1	65.6 → 27.0	621 → 44	−99.7 %	19.2 MiB → 0
O2	120.8 → 31.6	2624 → 69	−83.9 %	31.8 MiB → 0
O3	112.1 → 96.8	1031 → 1031	−19.7 %	0 → 0

5 ANALYSIS

5.1 Why the gain differs between tasks

Figure 8 compares the tasks. The gain depends on how much of the runtime the broadcast removes. In O1 the shuffled table is the full trip table, so wall time falls to 41 % of the baseline and executor run time to 42 % (2.4× speedup). In O3 the broadcast only affects the last stages: the three scans and the aggregation shuffles remain and dominate, so wall time falls by only 13.6 % (1.16×) while executor CPU time falls by 27.4 %. We did not separate the two likely causes of this gap: the extra jobs that build the broadcast tables (1 to 3), and the unchanged stages, which bound the speedup as in Amdahl's law. For O2 we only have the shuffle reduction (−83.9 %) and the vanished spill. Shuffle volume predicts the gain only when the shuffled table is the dominant cost: O1 (−99.7 % shuffle, −59 % wall time) versus O3 (−19.7 % shuffle, −14 % wall time).

5.2 Dataset scale and cluster size

All measurements use the 12 files of 2025 on one worker with 2 cores. We did not run the dataset-scale or cluster-size sweeps, so we

metric		
wall_time_s	65.64	27.03
num_jobs	1	5
num_stages	6	7
num_tasks	621	44
executor_run_time_ms	119529	50240
executor_cpu_time_ms	105540	37223
input_bytes	21.3MB	21.3MB
...		
shuffle_read_bytes	-99.7%	
shuffle_write_bytes	-99.7%	
memory_spilled_bytes	-100.0%	
disk_spilled_bytes	-100.0%	

metric		
wall_time_s	120.76	
num_jobs	3	
num_stages	19	
num_tasks	2624	
executor_run_time_ms	202713	
executor_cpu_time_ms	182414	
input_bytes	21.3MB	
...		
shuffle_read_bytes	4.6MB	-83.9%
shuffle_write_bytes	4.6MB	-83.9%
memory_spilled_bytes	0.0B	-
disk_spilled_bytes	0.0B	-

Figure 6: `profiler.compare` output for the optimised runs. Left: O1, baseline and optimised column, with the change for shuffle and spill. Right: O2, optimised run with the change for shuffle and spill.

metric	o3_demand_forecasting_features_before	o3_demand_forecasting_features_after	change
wall_time_s	112.07	96.79	-13.6%
num_jobs	1	3	+200.0%
num_stages	9	9	+0.0%
num_tasks	1031	1031	+0.0%
executor_run_time_ms	147894	116363	-21.3%
executor_cpu_time_ms	107477	78072	-27.4%
input_bytes	31.9MB	31.9MB	+0.0%
output_bytes	7.4MB	10.0MB	+35.2%
shuffle_read_bytes	126.7MB	101.7MB	-19.7%
shuffle_write_bytes	126.7MB	101.7MB	-19.7%
memory_spilled_bytes	0.0B	0.0B	-
disk_spilled_bytes	0.0B	0.0B	-

Figure 7: `profiler.compare` for O3: baseline (`before`) and broadcast variant (`after`).

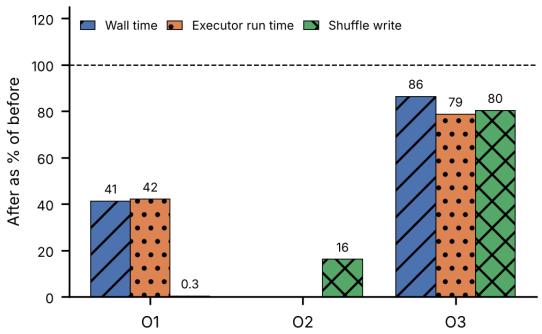


Figure 8: Effect of the broadcast joins as percentage of the baseline (100 % dashed). Bars for O2 wall time and executor time are missing because the baseline values are not available.

report no speedup or efficiency curve and make no scalability claim. The baseline suggests one expectation: with a median of 38 ms per shuffle task (45 % shuffle write time) and 10.6 MiB of input per scan, fixed per-task costs dominate, so more cores should help less than ideally. This remains a hypothesis until measured.

6 DISCUSSION

Remaining bottlenecks. The repeated scans are not fixed: O3 reads the trip table three times (31.9 instead of 10.6 MiB), and every task re-derives `clean_trips` from the raw files; writing the cleaned table once as Parquet or caching it is the obvious remedy. The 200 default shuffle partitions give tiny tasks (median 5.5 KiB read, 38 ms, 45 % shuffle write time); fewer partitions or AQE should reduce this overhead. The 4-week lag is still a self-join that shuffles the daily table twice; a window function over a day index could avoid it. O2 remains the slowest task (120.8 s) because of its window and Q1/Q3 stages. We identified these from plans and the UI but did not apply or measure them, so we claim no gain for them.

Limitations. The baseline deliberately disables AQE and automatic broadcast. Default Spark would already broadcast the 265-row lookup (10 MB threshold), so our gains overstate the benefit over a default configuration. The profiler stores one run per name, so all numbers are single measurements without variance, and file-cache effects are not controlled. The data are small (10.6 MiB per scan), the setup is one Docker worker, and the O3 output growth of 35.2 % is unexplained.

With more time. We would apply the remedies above one at a time with repeated runs and a warm-up, run the scale and cluster-size sweeps on the cluster, and compare against Spark's defaults (AQE and automatic broadcast on).

ACKNOWLEDGMENTS

We used the AI assistant Claude (Anthropic) and Gemini (Google) throughout the project: for drafts of the PySpark code (cleaning flags, window functions, O3 baseline), for debugging (Docker volume permissions, executor failures, Spark plans), and for drafting and editing the text of this report because English is not our native language. All measurements, screenshots, and conclusions come from our own runs, and we reviewed the text and code before submission.

Table 3: Individual Contributions

Team Member	Contributions / Work Division	Share (%)
Wiktor	Baseline implementation (Tasks M1–M3, Optional O1–O2), execution profiling, and measurements.	50%
Lukas	Baseline refinement, implementation of O3, report writing and structuring, performance evaluation.	50%

REFERENCES

[1] New York City Taxi and Limousine Commission. 2026. TLC Trip Record Data. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page. Yellow Taxi Trip Records, accessed November 2026.

[2] Prashant Shukla. 2024. *Spark enforceSchema option for Parquet.* https://medium.com/@shuklaprashant9264/spark-enforceschema-option-for-parquet-ac5c8a2a4a97

[3] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. 2016. Apache Spark: A Unified Engine for Big Data Processing. *Commun. ACM* 59, 11 (2016), 56–65.